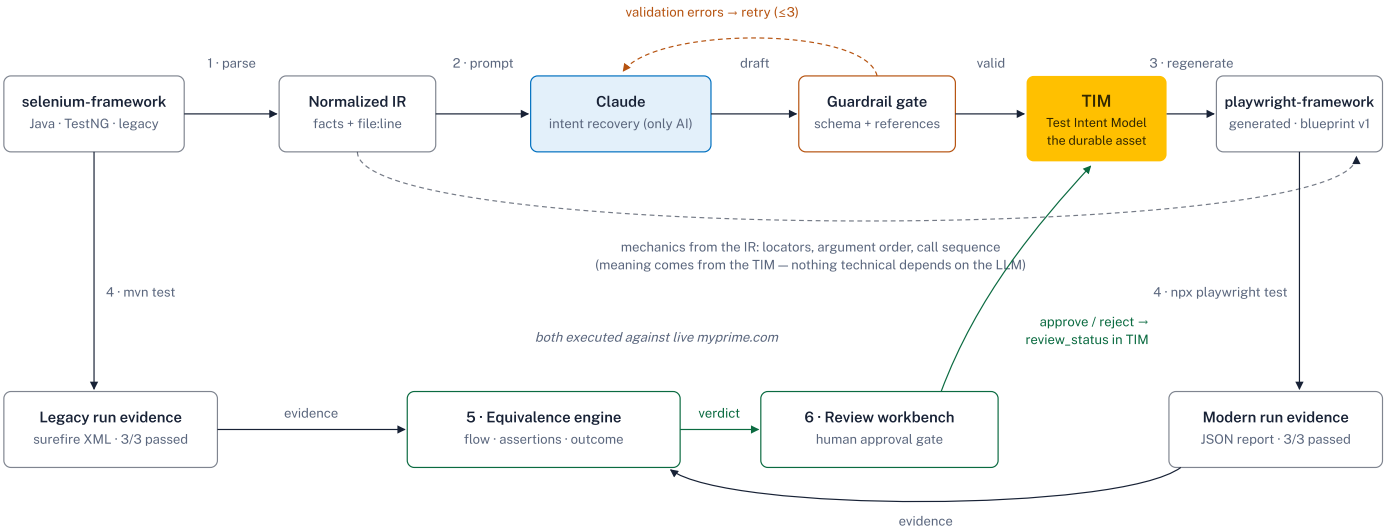
Test Intent Pipeline

The pipeline never translates Selenium code into Playwright code. It recovers **what each test means** into a framework-neutral Test Intent Model, regenerates a new implementation from that meaning under enterprise conventions, and then **proves** both implementations verify the same business behavior — with a human approving the result.

The closed loop

Six stages. AI touches exactly one of them — and its output must pass a deterministic guardrail gate before it becomes the Test Intent Model, the durable asset at the center.



The transformation chain (top) turns legacy code into a validated intent model and a regenerated framework; the validation chain (bottom) runs both against the real application and closes the loop by writing the human decision back into the TIM.

Six stages, one command each

The chips say who does the work. Parsing, generation, execution, and comparison are deterministic code; AI answers exactly one narrow question — "what does this test mean in business terms?" — and a human owns the final call.

1 Ingest

DETERMINISTIC

`python pipeline.py ingest`

A source adapter parses the Java framework — page objects, locators, action sequences, test methods, assertions, config, oracle data — into a **Normalized Technical IR** where every node carries its source file and line.

Real run: **4 page objects · 26 locators · 3 tests · 15 assertions** → `normalized_ir.json`

2 Understand

AI + VALIDATION

`python pipeline.py understand`

Claude recovers business intent from the IR as schema-conformant JSON — e.g. *"search for 'atorvastatin', select the matching suggestion, specify strength, quantity, frequency."* Validators reject any reference that doesn't resolve against the IR and feed the errors back for retry.

Real run: **3 TIM tests, avg confidence 0.88**; two ambiguous sanity checks honestly flagged at 0.6

3 Generate

DETERMINISTIC

`python pipeline.py generate`

Mechanics from the IR, meaning from the TIM: emits the whole Playwright framework under an enterprise blueprint — page objects, shared fixtures, data-driven oracles, and `TIM-001` traceability in every test title. Legacy timing quirks become named resilience patterns, not transliterated loops.

Real run: **25/25 methods regenerated** — 21 generic, 4 blueprint patterns, **0 TODO stubs**

4 Execute

DETERMINISTIC

`python pipeline.py execute`

Runs the legacy suite (`mvn test`) and the generated suite (`npx playwright test`) against the live application, normalizing both reports into per-test evidence: status, duration, failure detail.

Caught **2 real behavioral gaps** pre-approval: duplicate element ids, and a missing constructor load-gate

5 Compare

DETERMINISTIC

`python pipeline.py compare`

Explainable equivalence per test — never a bare pass/fail diff: is every intent step realized, is every business assertion regenerated against the same oracle, and did both implementations verify the same values live?

Real run: flow **3/3** · assertions **7/7** · outcomes match → **3× EQUIVALENT**

6 Review

HUMAN GATE

`streamlit run review_ui/app.py`

The workbench shows recovered intent with confidence and evidence, legacy vs generated code side by side, execution results, and the verdict's reasoning. Approve / Reject writes `review_status` back into the TIM.

Nothing AI-inferred becomes approved fact without a human decision.

Why the AI can't hallucinate a migration

GUARDRAIL · REFERENTIAL

Every claim must resolve

Each TIM step cites a `binding_ref` into the IR and each expected value a reference into the oracle data (`data:drugs.LIPITOR_BRAND.detailTierHeading`). Unresolvable output is rejected, not accepted.

GUARDRAIL · EVIDENTIAL

Confidence + evidence on every value

Every step and assertion carries a confidence score and `file:line` evidence back to source. Anything under 0.7 is flagged for human review instead of passing silently.

GUARDRAIL · PROVENANCE

The platform stamps, the human approves

Model, prompt version, and timestamp are stamped by platform code — never by the model. Approval exists only as a reviewer's action in the workbench.

The run, in numbers

3
TIM tests recovered from the legacy suite

0.88
average intent confidence, honestly scored

25/25
methods regenerated — zero TODO stubs

2
real behavioral gaps caught before approval

3/3
tests passing live in both frameworks

3×
verdict: EQUIVALENT — approve migration

> “We parse the legacy framework deterministically, ask AI one narrow question — **what does this test mean?** — validate its answer against source evidence, regenerate a modern framework from that meaning, run both against the real app, and hand a reviewer explainable proof they check the same behavior. **The intent model in the middle is framework-neutral** — the same TIM could generate Cypress or pytest tomorrow without re-reading a line of Selenium.”

```
selenium-framework → modernization-platform → playwright-framework · artifacts: ir / tim / runs / reports
```